Árboles de Decisión y Random Forest

Sesión Práctica - Aplicaciones en R

Elías Alvear
2025-08-27

Objetivos de Hoy

✅ **Implementar** árboles de decisión en casos reales

✅ **Interpretar** resultados y tomar decisiones de negocio

✅ **Optimizar** modelos usando pruning y validación cruzada

✅ **Comparar** árboles individuales vs Random Forest

✅ **Aplicar** feature importance para insights de negocio



1. Caso de Estudio: Detección de Spam
2. Ejercicio: Análisis de Crédito
3. Optimización con CP
4. Random Forest en Acción
5. Desafío Final: Caso Integrado

Actividad

En cada uno de los siguientes apartados, indique si la afirmacion es verdadera o falsa. En ambos casos, justifique.

- Si en un problema de clasificacion se observa que el rendimiento en el conjunto de validación es sustancialmente menor que en el conjunto de entrenamiento, puede haber problemas de sobreajuste.
- En general, en el proceso de construccion de un clasificador, mientras más datos de entrenamiento se utilicen, mejor sera el rendimiento del modelo resultante, en términos de sus capacidades de generalizacion.
- La tecnica de Random Forest consiste en construir una serie de arboles de decisión en que cada arbol considera un atributo diferente para el nodo raíz.

Caso 1: Sistema Anti-Spam

Vamos a desarrollar un **sistema de filtrado de emails** usando árboles de decisión.

```
# Cargar y explorar datos
library(DAAG)
data(spam7)
head(spam7)
```

```
##   crl.tot dollar  bang money n000 make yesno
## 1     278  0.000 0.778  0.00 0.00 0.00     y
## 2    1028  0.180 0.372  0.43 0.43 0.21     y
## 3    2259  0.184 0.276  0.06 1.16 0.06     y
## 4     191  0.000 0.137  0.00 0.00 0.00     y
## 5     191  0.000 0.135  0.00 0.00 0.00     y
## 6      54  0.000 0.000  0.00 0.00 0.00     y
```

Contexto de negocio:

- Empresa de email necesita filtro automático
- Falsos positivos = emails importantes en spam
- Falsos negativos = spam en bandeja de entrada
- **¿Cuál error es más costoso?**

Exploremos los Datos

```
# Resumen de variables
summary(spam7)
```

```
##     crl.tot            dollar             bang              money               n000
##  Min.   :    1.0   Min.   :0.00000   Min.   : 0.0000   Min.   : 0.00000   Min.   :0.0000
##  1st Qu.:   35.0   1st Qu.:0.00000   1st Qu.: 0.0000   1st Qu.: 0.00000   1st Qu.:0.0000
##  Median :   95.0   Median :0.00000   Median : 0.0000   Median : 0.00000   Median :0.0000
##  Mean   :  283.3   Mean   :0.07581   Mean   : 0.2691   Mean   : 0.09427   Mean   :0.1016
##  3rd Qu.:  266.0   3rd Qu.:0.05200   3rd Qu.: 0.3150   3rd Qu.: 0.00000   3rd Qu.:0.0000
##  Max.   :15841.0   Max.   :6.00300   Max.   :32.4780   Max.   :12.50000   Max.   :5.4500
##       make         yesno
##  Min.   :0.0000   n:2788
##  1st Qu.:0.0000   y:1813
##  Median :0.0000
##  Mean   :0.1046
##  3rd Qu.:0.0000
##  Max.   :4.5400
```

```
# Distribución de la variable objetivo
table(spam7$yesno)
```

```
##
##    n    y
```

Preparación de Datos

```
# División train/test
set.seed(123)
ind_train ← partition(1:nrow(spam7), p=c(0.7,0.3))
train ← spam7[ind_train$`1`,]
test ← spam7[ind_train$`2`,]

# Verificar distribución en ambos sets
cat("Train - Spam:", prop.table(table(train$yesno))["y"])
```

```
## Train - Spam: 0.3944099
```

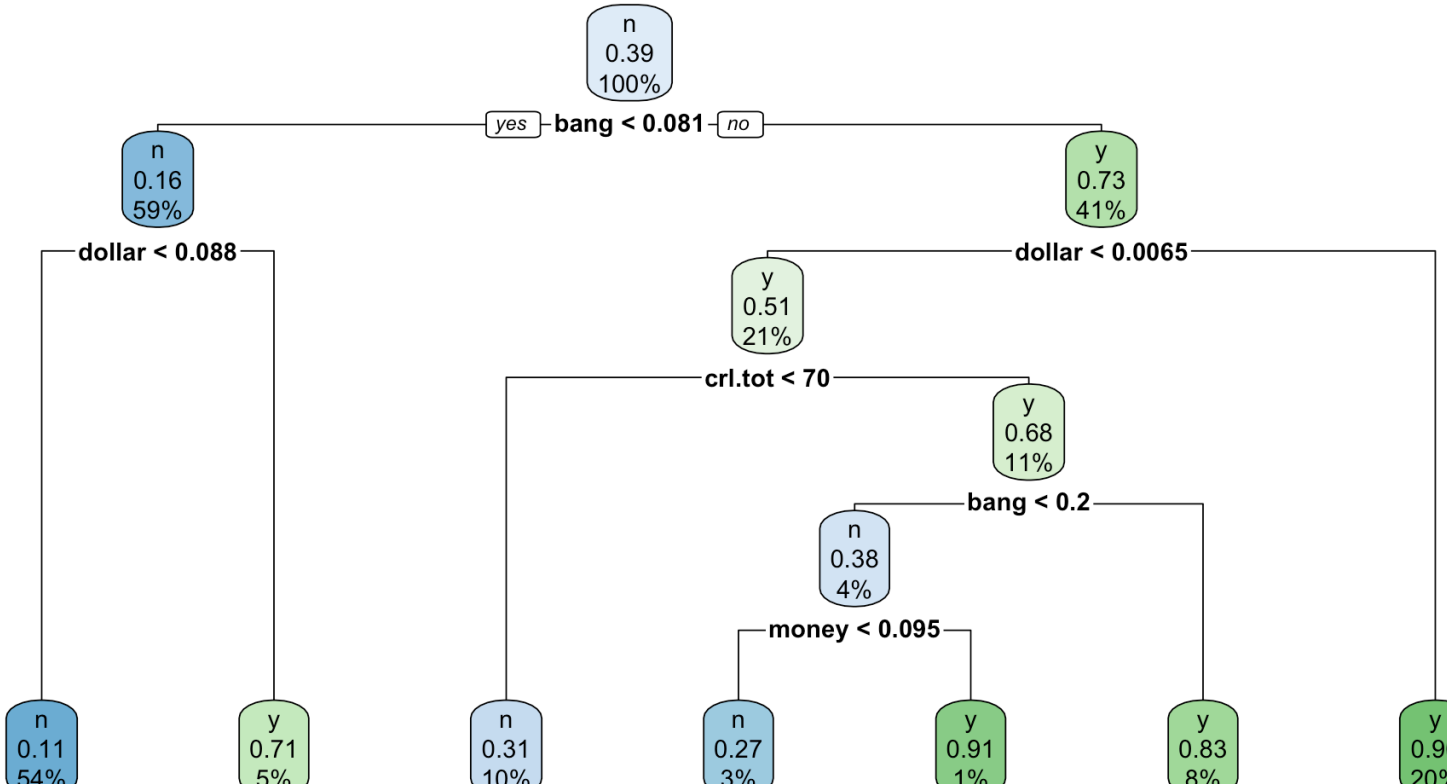
```
cat("Test - Spam:", prop.table(table(test$yesno))["y"])
```

```
## Test - Spam: 0.3931933
```

¿Por qué es importante verificar esto?

Nuestro Primer Árbol

```
# Árbol básico
tree_basic ← rpart(yesno ~ .,
                   data = train,
                   method = "class")
rpart.plot(tree_basic)
```



Interpretación del Negocio

Pregunta para discusión:

1. ¿Cuál es la primera variable que usa el algoritmo?
2. ¿Tiene sentido desde el punto de vista del negocio?
3. ¿Qué nos dice el tamaño de cada nodo?

```
# Información detallada del árbol
print(tree_basic)
```

```
## n= 3220
##
## node), split, n, loss, yval, (yprob)
##       * denotes terminal node
##
##  1) root 3220 1270 n (0.60559006 0.39440994)
##    2) bang< 0.0805 1885  294 n (0.84403183 0.15596817)
##      4) dollar< 0.0875 1728  182 n (0.89467593 0.10532407) *
##      5) dollar≥0.0875 157   45 y (0.28662420 0.71337580) *
##    3) bang≥0.0805 1335  359 y (0.26891386 0.73108614)
##      6) dollar< 0.0065 676  331 y (0.48964497 0.51035503)
##       12) crl.tot< 69.5 307   95 n (0.69055375 0.30944625) *
##       13) crl.tot≥69.5 369  119 y (0.32249322 0.67750678)
##         26) bang< 0.1985 125   48 n (0.61600000 0.38400000)
```

🎯 Ejercicio 1

1. **Crear** un árbol usando solo 3 variables que consideren más importantes
2. **Comparar** su performance con el árbol completo
3. **Preparar** una recomendación de negocio de 2 minutos

Evaluación de Performance

```r
# Predicciones
pred_train ← predict(tree_basic, train, type = "class")
pred_test ← predict(tree_basic, test, type = "class")

# Matriz de confusión - Test
confusionMatrix(pred_test, test$yesno, positive = "y")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   n   y
##          n 780 133
##          y  58 410
##
##                Accuracy : 0.8617
##                  95% CI : (0.8424, 0.8795)
##     No Information Rate : 0.6068
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.7029
##
##  Mcnemar's Test P-Value : 8.582e-08
##
##             Sensitivity : 0.7551
```

Análisis de Errores Críticos

```
# Crear matriz de confusión manual para análisis
conf_matrix ← table(Predicho = pred_test, Real = test$yesno)
print(conf_matrix)
```

```
##          Real
## Predicho   n   y
##        n 780 133
##        y  58 410
```

```
# Calcular costos específicos
falsos_positivos ← conf_matrix[2,1]  # Predice spam, es ham
falsos_negativos ← conf_matrix[1,2]  # Predice ham, es spam

cat("Falsos Positivos (Ham como Spam):", falsos_positivos)
```

```
## Falsos Positivos (Ham como Spam): 58
```

```
cat("Falsos Negativos (Spam como Ham):", falsos_negativos)
```

```
## Falsos Negativos (Spam como Ham): 133
```

Para el negocio: ¿Cuál tipo de error prefieren minimizar y por qué?

Optimización con Complexity Parameter

```
# Revisar tabla de CP
printcp(tree_basic)
```

```
##
## Classification tree:
## rpart(formula = yesno ~ ., data = train, method = "class")
##
## Variables actually used in tree construction:
## [1] bang    crl.tot dollar  money
##
## Root node error: 1270/3220 = 0.39441
##
## n= 3220
##
##         CP nsplit rel error  xerror     xstd
## 1 0.485827      0   1.00000 1.00000 0.021837
## 2 0.052756      1   0.51417 0.53228 0.018197
## 3 0.046063      2   0.46142 0.47087 0.017375
## 4 0.022835      4   0.36929 0.38189 0.015982
## 5 0.014173      5   0.34646 0.36220 0.015635
## 6 0.010000      6   0.33228 0.35906 0.015578
```

```
# Gráfico de CP
```

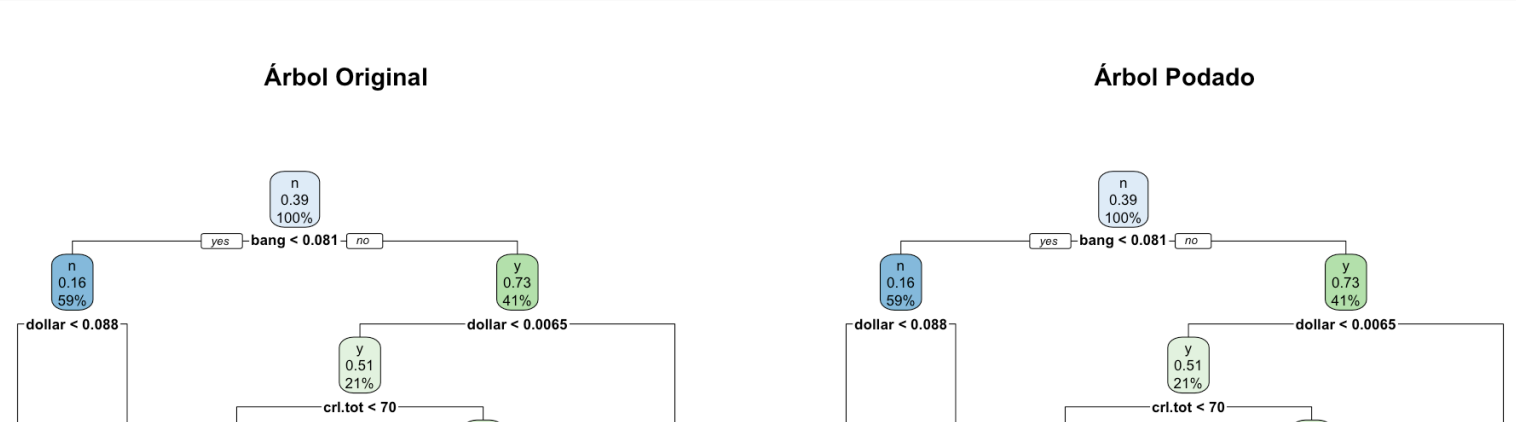
Encontrar CP Óptimo

```
# Encontrar CP que minimiza xerror
cp_optimo ← tree_basic$cptable[which.min(tree_basic$cptable[,"xerror"]),"CP"]
cat("CP óptimo:", cp_optimo)
```

```
## CP óptimo: 0.01
```

```
# Aplicar poda
tree_pruned ← prune(tree_basic, cp = cp_optimo)
```

```
# Comparar árboles
par(mfrow=c(1,2))
rpart.plot(tree_basic, main="Árbol Original")
rpart.plot(tree_pruned, main="Árbol Podado")
```



Performance del Árbol Podado

```
# Predicciones con árbol podado
pred_pruned ← predict(tree_pruned, test, type = "class")

# Comparar performance
conf_orig ← confusionMatrix(pred_test, test$yesno, positive = "y")
print(conf_orig$overall['Accuracy'])
```

```
##  Accuracy
## 0.8616944
```

```
conf_pruned ← confusionMatrix(pred_pruned, test$yesno, positive = "y")
print(conf_pruned$overall['Accuracy'])
```

```
##  Accuracy
## 0.8616944
```

Reflexión: ¿Siempre el árbol podado es mejor? ¿Por qué?

🌳 Random Forest Time!

Cuando un árbol no es suficiente

Random Forest para Spam

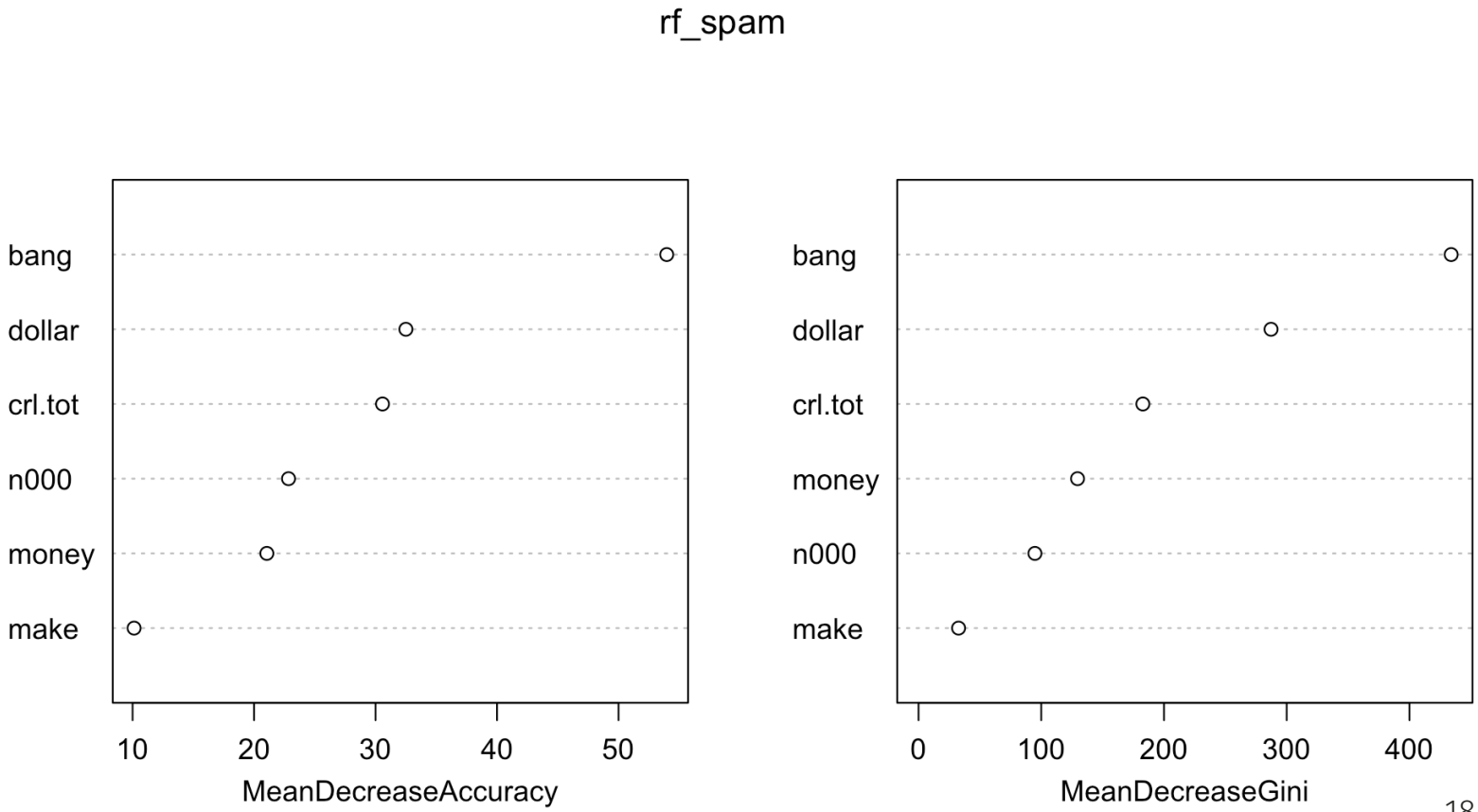
```r
# Random Forest
set.seed(2022)
rf_spam ← randomForest(yesno ~ .,
                       data = train,
                       ntree = 100,
                       importance = TRUE)

print(rf_spam)
```

```
##
## Call:
##  randomForest(formula = yesno ~ ., data = train, ntree = 100,      importance = TRUE)
##                Type of random forest: classification
##                      Number of trees: 100
## No. of variables tried at each split: 2
##
##         OOB estimate of  error rate: 11.61%
## Confusion matrix:
##      n    y class.error
## n 1834  116  0.05948718
## y  258 1012  0.20314961
```

Feature Importance

```r
# Gráfico de importancia
importance_data ← importance(rf_spam)
varImpPlot(rf_spam)
```



Comparación Final: Árbol vs Random

```r
# Predicciones Random Forest
pred_rf ← predict(rf_spam, test)

# Comparar todas las métricas
results ← data.frame(
  Model = c("Árbol Básico", "Árbol Podado", "Random Forest"),
  Accuracy = c(
    confusionMatrix(pred_test, test$yesno, positive = "y")$overall['Accuracy'],
    confusionMatrix(pred_pruned, test$yesno, positive = "y")$overall['Accuracy'],
    confusionMatrix(pred_rf, test$yesno, positive = "y")$overall['Accuracy']
  )
)
print(results)
```

```
##           Model  Accuracy
## 1  Árbol Básico 0.8616944
## 2  Árbol Podado 0.8616944
## 3 Random Forest 0.8783490
```

💰 Caso 2: Aprobación de Créditos

Un problema de negocio real

Dataset: German Credit

```r
# Cargar datos de crédito alemán
library(caret)
data(GermanCredit)

# Explorar estructura
str(GermanCredit)
```

```
## 'data.frame':    1000 obs. of  62 variables:
##  $ Duration                                : int  6 48 12 42 24 36 24 36 12 30 ...
##  $ Amount                                  : int  1169 5951 2096 7882 4870 9055 2835 6948
##  $ InstallmentRatePercentage               : int  4 2 2 2 3 2 3 2 2 4 ...
##  $ ResidenceDuration                       : int  4 2 3 4 4 4 4 2 4 2 ...
##  $ Age                                     : int  67 22 49 45 53 35 53 35 61 28 ...
##  $ NumberExistingCredits                   : int  2 1 1 1 2 1 1 1 1 2 ...
##  $ NumberPeopleMaintenance                 : int  1 1 2 2 2 2 1 1 1 1 ...
##  $ Telephone                               : num  0 1 1 1 1 0 1 0 1 1 ...
##  $ ForeignWorker                           : num  1 1 1 1 1 1 1 1 1 1 ...
##  $ Class                                   : Factor w/ 2 levels "Bad","Good": 2 1 2 2 1 2
##  $ CheckingAccountStatus.lt.0              : num  1 0 0 1 1 0 0 0 0 0 ...
##  $ CheckingAccountStatus.0.to.200          : num  0 1 0 0 0 0 0 1 0 1 ...
##  $ CheckingAccountStatus.gt.200            : num  0 0 0 0 0 0 0 0 0 0 ...
##  $ CheckingAccountStatus.none              : num  0 0 1 0 0 1 1 0 1 0 ...
##  $ CreditHistory.NoCredit.AllPaid          : num  0 0 0 0 0 0 0 0 0 0 ...
##  $ CreditHistory.ThisBank.AllPaid          : num  0 0 0 0 0 0 0 0 0 0 ...
```

Preparación del Caso Crédito

```r
# Simplificar para el ejercicio
credit ← GermanCredit %>%
  select(Class, Duration, Amount, InstallmentRatePercentage,
         ResidenceDuration, Age, NumberExistingCredits,
         NumberPeopleMaintenance, Telephone, ForeignWorker) %>%
  mutate(Class = as.factor(ifelse(Class == "Good", "Aprobado", "Rechazado")))

# División de datos
set.seed(456)
train_idx ← createDataPartition(credit$Class, p = 0.7, list = FALSE)
credit_train ← credit[train_idx, ]
credit_test ← credit[-train_idx, ]

head(credit)
```

```
##       Class Duration Amount InstallmentRatePercentage ResidenceDuration Age NumberExisti
## 1  Aprobado        6   1169                         4                 4  67
## 2 Rechazado       48   5951                         2                 2  22
## 3  Aprobado       12   2096                         2                 3  49
## 4  Aprobado       42   7882                         2                 4  45
## 5 Rechazado       24   4870                         3                 4  53
## 6  Aprobado       36   9055                         2                 4  35
##   NumberPeopleMaintenance Telephone ForeignWorker
## 1                       1         0             1
```

🎯 Ejercicio 2

Desafío: Desarrollar un sistema de aprobación de créditos

Entregables:

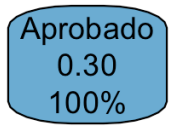
1. Árbol de decisión optimizado (usar CP)
2. Random Forest con feature importance
3. **Recomendación de negocio:** ¿Cuál modelo usarían y por qué?
4. **Insight de negocio:** 3 factores clave para aprobar créditos

Mi Solución: Árbol de Crédito

```
# Árbol básico
credit_tree ← rpart(Class ~ .,
                    data = credit_train,
                    method = "class")

# Encontrar CP óptimo
credit_cp ← credit_tree$cptable[which.min(credit_tree$cptable[,"xerror"]),"CP"]
credit_pruned ← prune(credit_tree, cp = credit_cp)
```

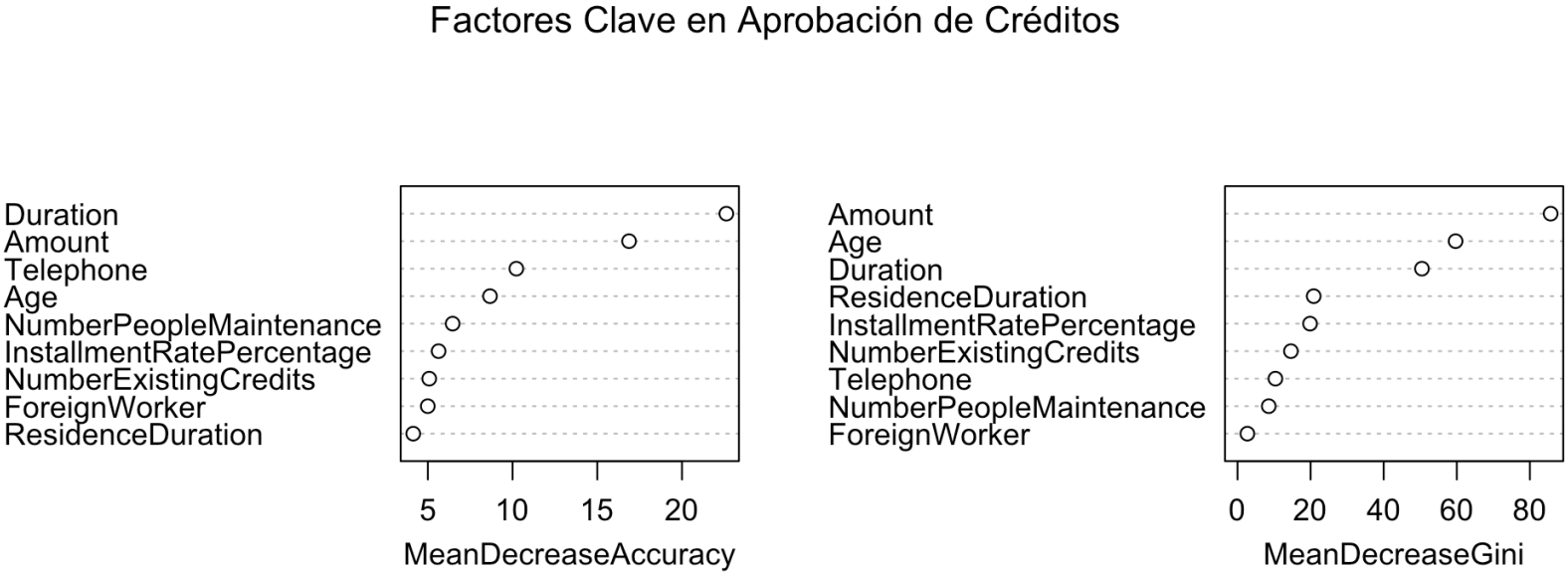
```
rpart.plot(credit_pruned, main = "Sistema de Aprobación de Créditos")
```



Random Forest para Créditos

```r
# Random Forest
credit_rf ← randomForest(Class ~ .,
                         data = credit_train,
                         importance = TRUE)

# Feature importance
varImpPlot(credit_rf, main = "Factores Clave en Aprobación de Créditos")
```



Comparación y Decisión de Negocio

```r
# Predicciones
pred_tree ← predict(credit_pruned, credit_test, type = "class")
pred_rf ← predict(credit_rf, credit_test)

# Métricas
tree_acc ← confusionMatrix(pred_tree, credit_test$Class)$overall['Accuracy']
rf_acc ← confusionMatrix(pred_rf, credit_test$Class)$overall['Accuracy']

# Matriz de confusión para análisis de costos
conf_tree ← confusionMatrix(pred_tree, credit_test$Class)
print(conf_tree$table)
```

```
##            Reference
## Prediction  Aprobado Rechazado
##   Aprobado       210        90
##   Rechazado        0         0
```

Pregunta crítica: En créditos, ¿qué es más costoso: rechazar un buen cliente o aprobar uno malo?

🚀 Desafío Final

Caso Integrado: Customer Churn

Dataset: Telecom Churn

```
##   MonthlyCharges TotalCharges Tenure InternetService       Contract    PaymentMethod Tec
## 1          75.48      4212.85      6               No       Two year      Credit card
## 2          19.78      3072.65     66            Fiber Month-to-month    Bank transfer
## 3          64.61      4474.83     23            Fiber       Two year      Credit card
## 4          68.66      3150.90     47              DSL       One year    Bank transfer
## 5          57.77      2348.44      1              DSL Month-to-month     Mailed check
## 6          55.31      1988.72     55              DSL       Two year Electronic check
##   StreamingTV OnlineBackup Churn
## 1         Yes           No    No
## 2         Yes          Yes   Yes
## 3          No          Yes    No
## 4         Yes          Yes    No
## 5          No          Yes    No
## 6          No           No    No
```

🎯 Desafío Final

Contexto: Empresa de telecomunicaciones pierde clientes

Su misión:

1. **Analizar** patrones de abandono con árboles de decisión
2. **Identificar** factores de riesgo con Random Forest
3. **Proponer** estrategia de retención basada en los hallazgos
4. **Calcular** ROI potencial de su estrategia

Estructura de Análisis Sugerida

```r
# 1. Exploración inicial
summary(churn_data)
prop.table(table(churn_data$Churn))

# 2. División de datos
# [Tu código aquí]

# 3. Árbol de decisión
# [Tu código aquí]

# 4. Random Forest
# [Tu código aquí]

# 5. Identificar segmentos de alto riesgo
# [Tu código aquí]

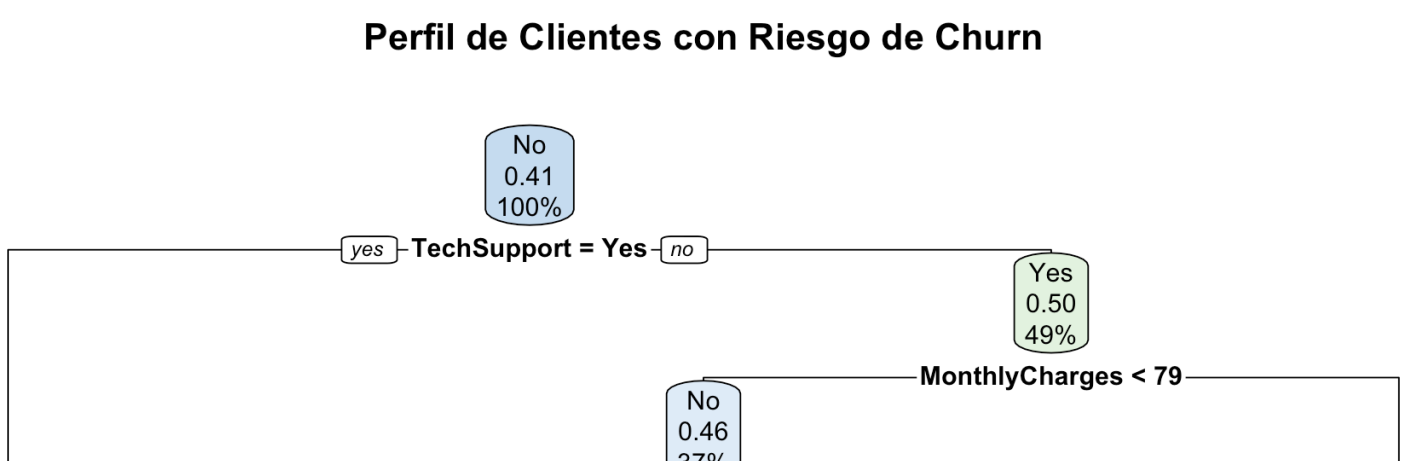
# 6. Propuesta de valor
# [Tu código aquí]
```

Mi Enfoque para Churn

```
# División de datos
set.seed(999)
churn_idx ← createDataPartition(churn_data$Churn, p = 0.7, list = FALSE)
churn_train ← churn_data[churn_idx, ]
churn_test ← churn_data[-churn_idx, ]

# Árbol de decisión
churn_tree ← rpart(Churn ~ ., data = churn_train, method = "class")
churn_tree_opt ← prune(churn_tree,
                       cp = churn_tree$cptable[which.min(churn_tree$cptable[,"xerror'

rpart.plot(churn_tree_opt, main = "Perfil de Clientes con Riesgo de Churn")
```



Insights de Random Forest

```r
# Random Forest
churn_rf ← randomForest(Churn ~ ., data = churn_train, importance = TRUE)

# Top factores de riesgo
importance(churn_rf) %>%
  as.data.frame() %>%
  arrange(desc(MeanDecreaseAccuracy)) %>%
  head()
```

```
##                          No         Yes MeanDecreaseAccuracy MeanDecreaseGini
## Contract         4.1138931  4.22448751             5.802680         19.26727
## TechSupport     -0.1074946  5.91387043             3.562841         14.61436
## OnlineBackup    -1.8305898 -0.31508625            -1.588608         12.39443
## MonthlyCharges  -3.2815385  0.03497241            -2.264944         83.74761
## StreamingTV     -2.8051799 -0.55037611            -2.598025         11.68944
## TotalCharges    -1.7862377 -2.53662386            -3.005568         75.96553
```

Segmentación para Estrategia

```r
# Identificar clientes de alto riesgo
pred_prob ← predict(churn_rf, churn_test, type = "prob")
churn_test$ChurnProb ← pred_prob[,"Yes"]

# Segmentos de riesgo
churn_test$RiskSegment ← cut(churn_test$ChurnProb,
                             breaks = c(0, 0.3, 0.7, 1),
                             labels = c("Bajo", "Medio", "Alto"))

table(churn_test$RiskSegment)
```

```
##
##  Bajo Medio  Alto
##    70   222     7
```

Pregunta estratégica: ¿Cómo personalizarían la estrategia de retención para cada segmento?

ROI de la Estrategia

```r
# Supuestos de negocio
clv ← 2000  # Customer Lifetime Value promedio
retention_cost ← c(50, 100, 200)  # Costo por segmento
retention_rate ← c(0.1, 0.3, 0.6)  # Tasa de éxito por segmento

# Calcular ROI por segmento
segments ← table(churn_test$RiskSegment)
roi_analysis ← data.frame(
  Segmento = names(segments),
  Clientes = as.numeric(segments),
  CostoRetencion = retention_cost,
  TasaExito = retention_rate,
  ClientesRetenidos = as.numeric(segments) * retention_rate,
  IngresoRetenido = as.numeric(segments) * retention_rate * clv,
  CostoTotal = as.numeric(segments) * retention_cost,
  ROI = (as.numeric(segments) * retention_rate * clv - as.numeric(segments) * retentic
)

print(roi_analysis)
```

```
##   Segmento Clientes CostoRetencion TasaExito ClientesRetenidos IngresoRetenido CostoTota
## 1     Bajo       70             50       0.1                7.0           14000       350
## 2    Medio      222            100       0.3               66.6          133200      2220
## 3     Alto        7            200       0.6                4.2            8400       140
```

Lo que Aprendimos Hoy

Técnico:

- Árboles de decisión son interpretables pero inestables
- CP es crucial para evitar overfitting
- Random Forest mejora predicción pero pierde interpretabilidad
- Feature importance revela drivers de negocio

Negocio:

- Diferentes errores tienen diferentes costos
- La interpretabilidad a veces vale más que la precisión
- Los modelos deben alinearse con la estrategia
- ROI debe guiar las decisiones de implementación

Cuándo Usar Cada Método

Árboles de Decisión:

- ✅ Necesitas explicar decisiones (regulaciones, medicina)
- ✅ Stakeholders no-técnicos
- ✅ Prototipado rápido
- ❌ Máxima precisión es crítica

Random Forest:

- ✅ Precisión es prioridad
- ✅ Datos complejos con muchas variables
- ✅ Feature importance es suficiente
- ❌ Necesitas explicar decisiones individuales

Propuesta de estudio

Preparación:

1. Revisar conceptos de ensemble methods
2. Leer sobre XGBoost y Gradient Boosting
3. Pensar en un proyecto personal donde aplicar estos métodos

Recursos adicionales:

- Interpretable ML Book
- Random Forest Paper
- LIME for model interpretation

¡Gracias!

¿Preguntas?

📧 eliasalvearb@gmail.com

💼 LinkedIn: https://www.linkedin.com/in/elias-alvear-binimelis/